



向线程池提交任务（中）

北京理工大学计算机学院
金旭亮

Callable接口

Callable接口定义的call方法有返回值，并且声明了抛出Exception异常。

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Callable接口是一个函数式接口，因此，它定义的变量，可以接收一个Lambda表达式。

Callable任务的提交

ExecutorService接口定义有一个submit方法，用于将一个Callable对象发送到线程池的任务队列中等待执行：

```
<T> Future<T> submit(Callable<T> task);
```

注意一下上述方法返回一个Future对象，任务提交者可以通过它来取回结果。

Future接口

```
public interface Future<V> {  
    boolean cancel(boolean mayInterruptIfRunning);  
    boolean isCancelled();  
    boolean isDone();  
    V get() throws InterruptedException, ExecutionException;  
    V get(long timeout, TimeUnit unit)  
        throws InterruptedException, ExecutionException,  
            TimeoutException;  
}
```

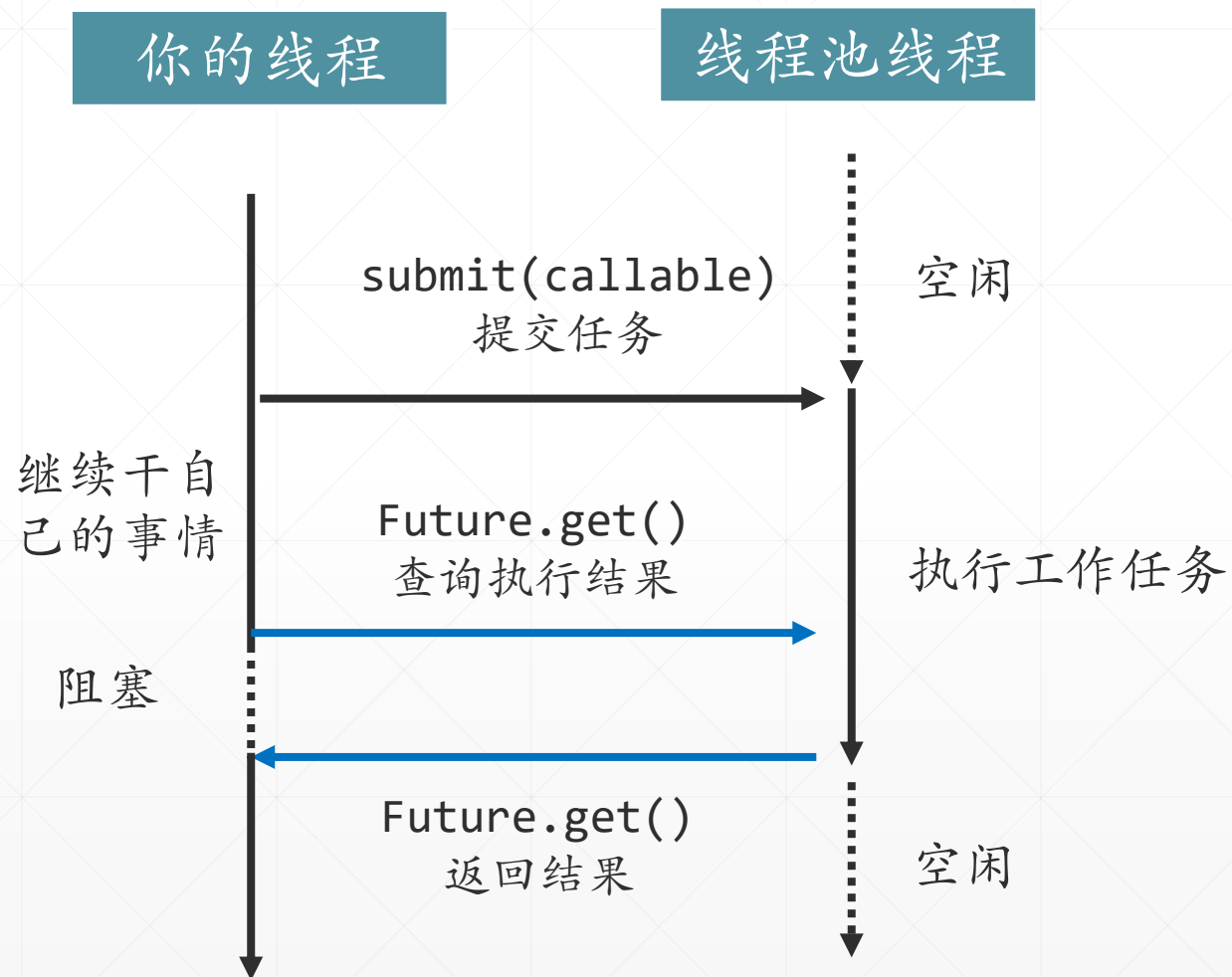
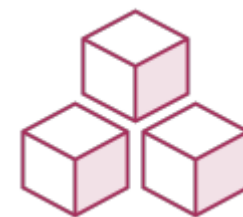
Future接口代表一个异步操作的结果。这个结果只能通过get()方法取得，如果调用此方法时异步操作还没有完成，调用者线程将会阻塞等待，直到取出结果才返回。

理解Future对象



- ✓ Future代表“当前没有，但未来会有”的一个值。
- ✓ 在未来的“某个时刻”，调用Future的get方法，就可以得到这个值。
- ✓ 调用Future的get方法，会阻塞调用者线程。

基于Future的异步编程模式



Callable与Future实例-1

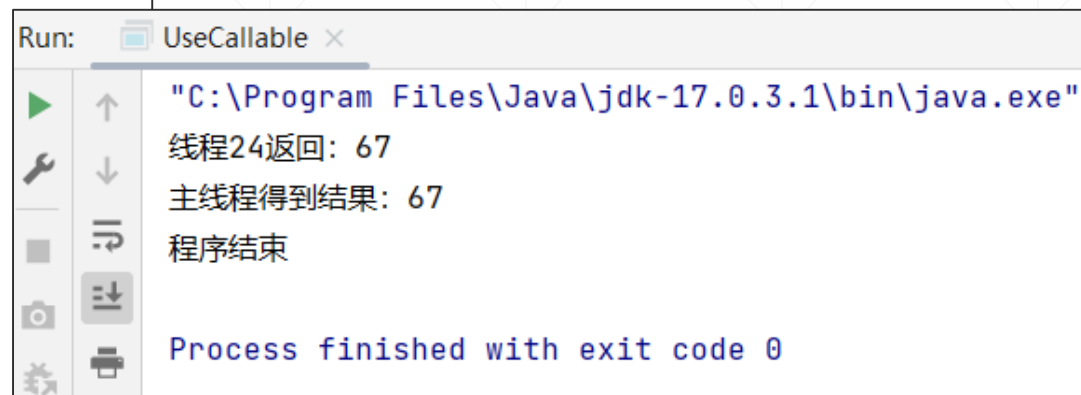
```
//使用Callable封装可以返回结果的多线程代码
class MyCallableClass implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        Random random = new Random();
        int ranValue = random.nextInt( bound: 100);
        int sleepTime = random.nextInt( bound: 3000);
        Thread.sleep(sleepTime);
        var threadId :long = Thread.currentThread().getId();
        System.out.println("线程" + threadId + "返回: " + ranValue);
        return ranValue;
    }
}
```

UseCallable.java

Callable与Future实例-2

```
public class UseCallable {  
    public static void main(String[] args)  
        throws InterruptedException {  
        //创建线程池  
        var es : ExecutorService = Executors.newFixedThreadPool( nThreads: 2);  
        //提交异步任务  
        var future : Future<Integer> = es.submit(new MyCallableClass());  
        //取回结果  
        try {  
            int value = future.get();  
            System.out.println("主线程得到结果: " + value);  
        } catch (ExecutionException e) {  
            System.out.println("捕获到异常: " + e.getMessage());  
        }  
        System.out.println("程序结束");  
    }  
}
```

运行截图



Run: UseCallable x

"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"

线程24返回: 67
主线程得到结果: 67
程序结束

Process finished with exit code 0



Callable接口

代表一个将被线程运行的任务，有返回值，可以抛出异常

只有一个成员——`call()`方法，它返回一个Future对象。

Future接口

代表一个将由线程池执行的任务

接口中最重要的方法是`get()`

- 此方法将阻塞等待任务完成
- 将返回一个Callable接口中指定类型的结果
- 可能抛出Callable接口所定义的异常